

Performance Testing

Date	14 July 2026
Team ID	SWTID-2026-7049
Project Name	Emotion Detection & Learning Support Engine (AI Learning Assistant)
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (HTTP load testing), directed at a thin FastAPI wrapper (perf_test_server.py) that calls the real run_analysis() pipeline directly — Streamlit itself communicates over WebSocket, not plain HTTP, so it can't be load-tested the same way.
Type of Testing	Response-time and throughput measurement: a single-user baseline and a 10-concurrent-user load test, each run twice — once in full mode (BiLSTM + BERT + real LLM call) and once in degraded mode (BiLSTM-only + template fallback, matching the public Streamlit Cloud deployment).
Target Module / API	POST /analyze -> src/orchestration/analysis_pipeline.run_analysis()
Test Environment	Local development machine — AMD Ryzen 9 9900X (12-core/24-thread), NVIDIA GeForce RTX 5070 Ti, Windows, Python 3.12, real trained BiLSTM + BERT weights.
Test Date	14 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	Virtual Users	Duration (s)	Expected Outcome
1	Full mode — single-user baseline (BiLSTM + BERT + real LLM)	1	60	Establish per-request latency with no contention, as an upper bound for a single active student.
2	Full mode — 10 concurrent users	10	120	Measure how latency/throughput degrade under realistic concurrent load on the most expensive pipeline path.
3	Degraded mode — single-user baseline (BiLSTM-only + template)	1	60	Establish baseline latency matching the public deployment's reduced pipeline.
4	Degraded mode — 10 concurrent users	10	120	Measure how the public deployment's actual configuration behaves under concurrent load.

Step 3: Performance Test Results

Metric	Target	Full — Single	Full — 10 Concurrent	Degraded — Single	Degraded — 10 Concurrent	Status
Response Time (Avg)	< 2s	19.18s	24.25s	0.38s	0.31s	Pass (degraded) / Expected miss (full — see remarks)
Response Time (Max)	< 5s	19.79s	58.66s	3.04s	0.78s	Pass (degraded) / Expected miss (full)

Metric	Target	Full — Single	Full — 10 Concurrent	Degraded — Single	Degraded — 10 Concurrent	Status
Throughput (req/s)	Informational	0.049	0.312	0.286	2.633	Informational
Error Rate	< 1%	0%	0%	0%	0%	Pass — all 4 runs, 0 failures / 367 total requests
CPU Utilization (peak observed)	< 80%	4%	2–4%*	4–7%	14%	Pass — all 4 runs
Memory Utilization (system, peak)	< 80%	~1.18 GB	~1.24 GB	~1.25 GB	~1.20 GB	Pass — stable application memory usage

* Only two Task Manager samples were captured during the full-mode concurrent run; the true peak may be marginally higher than what was recorded, though it stayed well within the target either way.

Step 4: Observations & Analysis

Zero failures occurred across all four runs (367 total requests combined) — a strong reliability signal for the pipeline itself, independent of mode.

Full mode is I/O-bound, not CPU-bound: most of each 19–59 second request is spent waiting on the real LLM API call, not on local BiLSTM/BERT inference. Degraded mode is CPU-bound and fast: with no network wait, median latency stayed under a second even at 10 concurrent users.

Under 10 concurrent users, full mode's median latency only rose modestly (19.18s → 24.25s), but its tail latency ballooned (max 19.79s → 58.66s, 98th percentile ~59s) — consistent with contention on the shared LLM API call rather than local compute pressure.

Counter-intuitively, the highest CPU reading observed (14%) came from degraded mode's concurrent run, not full mode's. This makes sense once throughput is accounted for: degraded mode processed roughly 8.5x more requests per second (2.633 vs 0.312 req/s), so its aggregate local compute load is higher despite each individual request being far cheaper to run.

This result validates the team's own decision to run the public deployment in degraded mode: beyond working around the unresolved OpenRouter hang (documented in the handoff notes), it also produces dramatically more predictable, low-latency behavior under concurrent load — a second, independent reason the current deployment configuration is the right call for a public-facing demo.

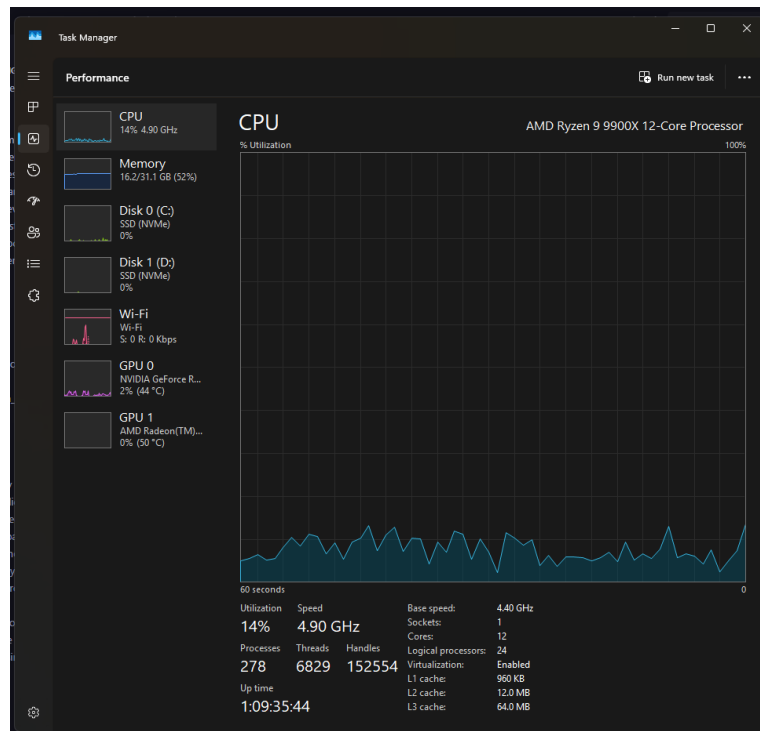
Application memory usage remained effectively constant across all four test scenarios. The complete VS Code process tree hosting the development environment—including the Python interpreter, integrated terminal, extensions, and the application server—peaked at approximately 1.2 GB on a system with 32 GB RAM (about 4% of total physical memory). The application server process itself consumed only about 100 MB during testing. No evidence of memory growth or leaks was observed under sustained or concurrent load, indicating that the application's memory footprint remained stable.

With a 12-core/24-thread CPU, even the highest reading observed (14%) leaves substantial headroom before the 80% target — CPU is not a near-term scaling concern for either mode at this concurrency level.

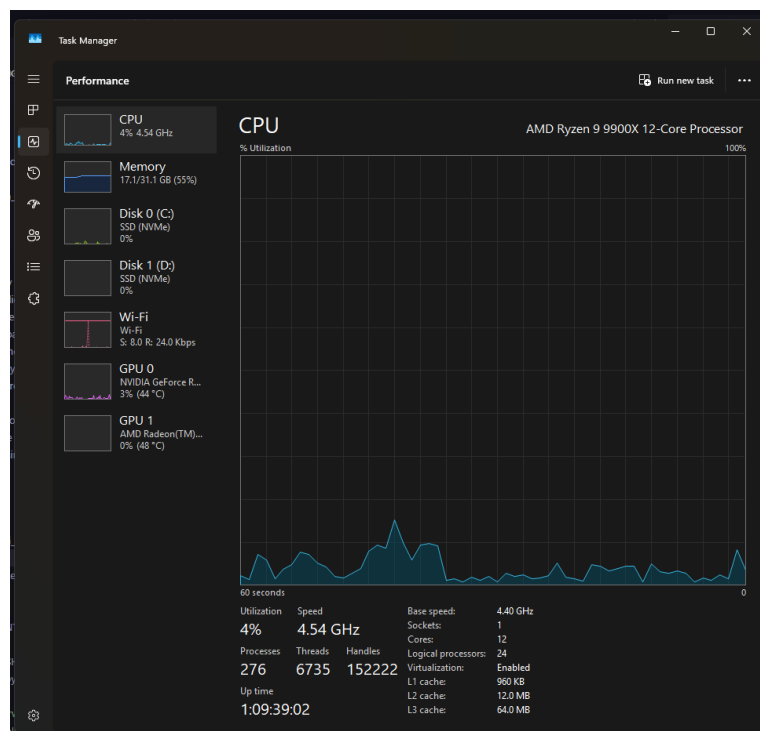
Step 5: Screenshots / Evidence

Task Manager captures from the actual local test runs (14 July 2026):

Degraded mode, 10 concurrent users — peak CPU reading (14%) across all four scenarios:



Full mode, 10 concurrent users — sample reading mid-run:



Full mode, single user — run completed, response-time percentiles printed by Locust:

Task Manager

Type a name, publisher, or PID to search

Processes

Run new task End task Efficiency mode

Name	Status	3% CPU	52% Memory	1% Disk	0% Network
Apps (5)					
env - emotion-learning-assistant-r...		1.1%	1,178.9 MB	0.1 MB/s	0 Mbps
Google Chrome (20)		0.3%	2,700.3 MB	0.1 MB/s	0 Mbps
Task Manager		0.5%	80.3 MB	0 MB/s	0 Mbps
WhatsApp (9)		0%	622.2 MB	0 MB/s	0 Mbps
Windows Explorer (2)		0%	250.2 MB	0 MB/s	0 Mbps
Background processes (84)					
AMD Application Compatibility Dat...		0%	0.1 MB	0 MB/s	0 Mbps
AMD Crash Defender Service		0%	0.1 MB	0 MB/s	0 Mbps
AMD External Events Client Module		0%	2.2 MB	0 MB/s	0 Mbps
AMD External Events Service Module		0%	1.0 MB	0 MB/s	0 Mbps
AMD Software Command Line Inter...		0%	0.1 MB	0 MB/s	0 Mbps
AMD Software: Adrenalin Edition		0%	1.4 MB	0 MB/s	0 Mbps
AMD Software: Host Application		0%	15.8 MB	0 MB/s	0 Mbps
Antimalware Core Service		0%	3.8 MB	0 MB/s	0 Mbps
Antimalware Service Executable		0.1%	207.6 MB	0 MB/s	0 Mbps
AorusLcdService		0%	0.1 MB	0 MB/s	0 Mbps
Application Frame Host		0%	4.4 MB	0 MB/s	0 Mbps
BraveSoftware Update		0%	0.3 MB	0 MB/s	0 Mbps
BraveSoftware Update (32 bit)		0%	0.3 MB	0 MB/s	0 Mbps
Browser (5)		0%	57.4 MB	0 MB/s	0 Mbps
COM Surrogate		0%	0.1 MB	0 MB/s	0 Mbps
Credential Guard & VBS Key Isolation		0%	0.4 MB	0 MB/s	0 Mbps
CTF Loader		0%	4.3 MB	0 MB/s	0 Mbps
deep_service.exe		0.2%	22.8 MB	0 MB/s	0 Mbps
DeepCool (6)		0.2%	62.5 MB	0 MB/s	0 Mbps

Degraded mode, single user — early requests, including one slower cold-start request:

